

Наследование и runtime polymorphism

Задание

Разработать программу для расчета парковочных сессий платной парковки

Парковка принимает легковые автомобили, мотоциклы, микроавтобусы и грузовые автомобили. Для разных типов транспорта отличаются стоимость парковки, количество занимаемых мест и допустимые зоны

Использовать следующие зоны парковки:

Зона	Назначение
A	обычная открытая парковка
B	крытая парковка
C	грузовая зона

Использовать следующие правила:

Тип транспорта	Занимаемые места	Допустимые зоны	Тариф
легковой автомобиль	1	A, B	120 рублей за час
мотоцикл	1	A, B	60 рублей за час
микроавтобус	2	B	180 рублей за час
грузовой автомобиль	3	C	250 рублей за час + 40 рублей за тонну груза за час

Длительность парковки считать в минутах. Для расчета стоимости неполный час оплачивается как полный. Минимальная оплачиваемая длительность - 1 час

Исходные данные находятся в файле parking_log.txt. Файл содержит события въезда и выезда транспорта

Строка въезда имеет формат:

```
ENTER время номерной_знак тип_транспорта зона
```

Для грузового автомобиля строка въезда содержит вес груза в тоннах:

```
ENTER время номерной_знак TRUCK зона вес_груза
```

Строка выезда имеет формат:

```
EXIT время номерной_знак
```

Использовать следующие значения типа транспорта:

Значение	Тип транспорта
CAR	легковой автомобиль
MOTORCYCLE	мотоцикл
MINIBUS	микроавтобус
TRUCK	грузовой автомобиль

Например:

```
ENTER 08:10 A123BC CAR A
ENTER 09:00 M777MM MOTORCYCLE B
EXIT 09:40 M777MM
EXIT 10:05 A123BC
ENTER 11:20 B456TY MINIBUS B
EXIT 14:05 B456TY
ENTER 07:30 T900TT TRUCK C 8
EXIT 12:10 T900TT
ENTER 13:00 C111CC CAR C
EXIT 14:00 C111CC
ENTER 15:15 T222AA TRUCK B 5
EXIT 16:45 T222AA
```

Каждое событие ENTER начинает парковочную сессию. Событие EXIT завершает активную сессию по номерному знаку. Формат строк можно считать корректным. Для каждого въезда в файле есть соответствующий выезд

Для транспорта создать общий абстрактный интерфейс. Конкретные типы транспорта должны реализовывать этот интерфейс через public inheritance

Парковочная сессия должна содержать транспортное средство и данные конкретного посещения парковки. Связь сессии с транспортом должна быть построена так, чтобы программа могла хранить завершённые сессии для разных типов транспорта в одной коллекции и обрабатывать их единообразно

Контейнер сессий должен владеть транспортными средствами полиморфно через `std::unique_ptr` на базовый тип. Удаление объектов через базовый тип должно работать корректно

Для каждой сессии программа должна определить:

- допустима ли выбранная зона для этого транспорта
- длительность парковки в минутах
- количество оплачиваемых часов
- количество занимаемых парковочных мест
- итоговую стоимость

Недопустимые сессии должны попадать в отчет с причиной ошибки. Стоимость для недопустимых сессий считать не требуется

Программа должна вывести:

- отчет по каждой сессии
- общую выручку по допустимым сессиям
- общее количество занятых парковочных мест по допустимым сессиям
- количество недопустимых сессий

Полноценный интерактивный CLI реализовывать не требуется. Программа должна читать события из файла `parking_log.txt`

Требования

- Названия переменных, функций, типов и других сущностей программы должны быть осмысленными и записываться на английском языке без транслитерации
- В программе должен быть общий абстрактный интерфейс транспорта с virtual функциями
- Для каждого типа транспорта должен быть отдельный производный класс с override
- Базовый полиморфный тип должен иметь virtual destructor
- Коллекция сессий должна хранить транспортные средства через std::unique_ptr на базовый тип
- Программа должна читать события парковки из файла parking_log.txt
- В программе должны быть примеры работы через ссылку или указатель на базовый тип
- В основной модели парковки нельзя хранить производные объекты как значения базового типа
- Программа должна использовать composition для связи парковочной сессии с транспортом и параметрами посещения
- Программа должна быть грамотно разбита на модули и содержать как минимум один .hpp файл и два .cpp файла. Допускается использовать больше файлов
- Для заголовочных файлов использовать #pragma once
- Программа должна компилироваться и выполняться без ошибок
- Студент должен понимать написанный код, уметь обосновать свое решение и отвечать на вопросы во время защиты лабораторной работы

Минимальный набор операций:

- проверка допустимости зоны для транспорта
- чтение события парковки из входного потока
- расчет длительности парковочной сессии
- расчет стоимости парковки через общий интерфейс транспорта